



WEB SCRAPING & REGRESSION ANALYSIS USING ASOS

RESEARCH QUESTION:

**DO PRODUCTS MARKETING AS SUSTAINABLE
HAVE HIGHER PRICES ON ASOS?**

MOTIVATION:

- **INTEREST IN FASHION, SUSTAINABILITY, AND MARKETING**
- **SUSTAINABILITY MAY ACT AS A MARKETING SIGNAL**
- **BRANDS MAY USE SUSTAINABILITY TO JUSTIFY HIGHER PRICES**
- **CAN HELP COMPANIES WITH MARKETING STRATEGIES**

TRU ANNAFI – ECO590



earth paying the
price for fast fashion



you paying the
price for ethically
made eco-friendly
products

 strictly fantastic

WEB SCRAPING PROCESS

- DATA COLLECTED FROM ASOS WEBPAGES

SCRAPED:

- PRODUCT NAMES
- PRICES
- PRODUCT LINKS

USED:

- REQUESTS - SENDS REQUESTS TO WEBSITES, DOWNLOADS WEBPAGES, API'S ETC.
- BEAUTIFULSOUP - HELPS READ AND ORGANIZE HTML SO WE CAN GET ITS DATA
- PANDAS - WORKS WITH THE DATASETS(TABLES/MERGING/CLEANING, ETC)

HTML TAGS USED:

- `<LI>` PRODUCT TILES
- `<P>` PRODUCT DESCRIPTIONS
- ARIA-LABEL FOR PRICES
- `<A>` TAGS FOR PRODUCT LINKS
- ADDED RANDOM DELAYS AND USER-AGENTS TO REDUCE BLOCKING

`<LI>PRODUCTTILE`

`<A>PRODUCT LINKS`

`<P>PRODUCTDESCRIPTION`

```
<li id="product-210490832" class="productTile_U0c1N">
  <a class="productLink_KM4PI" href="https://www.asos.co
  m/us/topshop/topshop-cape-draped-tee-in-white/prd/21049
  0832#colourWayId-210490834" aria-label="Topshop cape dr
  aped tee in white, Price $60.00">
    <div class="productMediaContainer_kmkXR mediaContaine
    r_rdzv9">
      <div class="productInfo_rwyH5" aria-label="Topshop ca
      pe draped tee in white, Price $60.00" tabindex="0"
      role="region">
        <p id="pta-product-210490832-0" class="productDescri
        ption_sryaw">Topshop cape draped tee in white</p>
        <p id="pta-product-210490832-1" aria-hidden="true"
        aria-label="Price: $60.00" class="container_WYZEU">
```

ARIA LABEL, PRICE

INSPECT PAGE SNIPPET

VARIABLE CONSTRUCTION

VARIABLES COLLECTED:	• PRICE • PRODUCT NAME • PRODUCT LINK
----------------------	---

VARIABLES CREATED:	SUSTAINABLE BINARY VARIABLE • 1 = CIRCULAR DESIGN COLLECTION • 0 = STANDARD ASOS PRODUCTS
--------------------	---

CATEGORY
• CREATED USING KEYWORD MATCHING • CATEGORIES: • TOPS • JEANS • DRESSES • SHORTS • OTHER

ASOS DID NOT DIRECTLY PROVIDE A SUSTAINABILITY VARIABLE, I CONSTRUCTED IT BASED ON WHETHER PRODUCTS BELONGED TO THE CIRCULAR DESIGN COLLECTION.

```
#Category Keywords (looked through all the items and saw what words were being used,
category_map = {
    "tops": ["top", "t-shirt", "tee", "shirt", "cami", "halter", "blouse", "tank"],
    "jeans": ["jeans", "jean", "pants", "trousers"],
    "dress": ["dress", "gown"],
    "shorts": ["shorts", "short", "jorts"]}
```

```
#Category Function. Sorts the items into the right category
def get_category(name): #creating a get_category function
    if not name: #checks if the name is empty/missing
        return "other" #will be categorized as other
    name = name.lower().replace("topshop", "") #"lower" turns everything lower
    for category, keywords in category_map.items(): #This loops through the c
        if any(re.search(rf"\b{word}\b", name) for word in keywords): #this s
            return category #if something matched, add it to that category
    return "other" #if nothing matched say its other
```

```
#All the Webpages We're Getting Data From
sources = [
    {"name": "tops", "url": "https://www.asos.com/women/tops/cat/?cid=4169&page=", "pages": 5, "sustainable": 0}, #the pages for how many numbers to
    {"name": "jeans", "url": "https://www.asos.com/us/women/jeans/cat/?cid=3630&page=", "pages": 5, "sustainable": 0}, #sustainable is equal to 0 be
    {"name": "shorts", "url": "https://www.asos.com/us/search/?q=shorts&page=", "pages": 5, "sustainable": 0},
    {"name": "dresses", "url": "https://www.asos.com/us/search/?q=dress&page=", "pages": 5, "sustainable": 0},
    {"name": "circular", "url": "https://www.asos.com/us/search/?q=circular+design+collection&page=", "pages": 2, "sustainable": 1}] #Less pages to
```

```
#Main Loop - goes through all the ASOS sections, scrapes the data, stores the info all together
all_data = [] #created an empty List for all the data
for source in sources: #Loop through every item in the sources List
    print(f"\nScraping {source['name']}...") #prints a progress message. "source['name']" this gets the name value in the dictionary. "\n" makes a new L
    df = scrape_asos(base_url=source["url"], pages=source["pages"], sustainable_flag=source["sustainable"]) #calls the scraping function which was made
    all_data.append(df) #adds the dataframes to "all_data". append() adds something to a list. so this dataframe list gets longer with append
```

```

#Scraping Data Function
def scrape_asos(base_url, pages, sustainable_flag): #base url is the site I'm web scraping, pages is how many pages I'll be looping through, sustainable
    products = [] #made an empty list to store all of the scraped data
    session = requests.Session() #using the library (requests) imported earlier. Session keeps a stable connection, cookies get saved, Login is saved, e

    for page in range(1, pages + 1): #Loops through pages
        url = base_url + str(page) #builds urls
        success = False

        for attempt in range(3): #retries the loop up to 3 times because it can fail
            try:
                headers = {"User-Agent": random.choice(user_agents)} #picks a random browser identity to also lessen the likelihood of being blocked
                response = session.get(url, headers=headers, timeout=20) #sending a request(session.get) to the website we want to visit(url), timeout w
                soup = BeautifulSoup(response.text, "html.parser") #takes the html from website response, html parser organizes it, it's then stored in
                items = soup.find_all("li", class_=lambda x: x and "productTile" in x) #finds all elements that have "productTile" somewhere in the name
                if not items: #if "productTile" isn't there (if its empty python will see it as false) from the items collected in the line above, then.
                    raise Exception("No items found") #raise forces an error, exception is an error type that lets us make a custom error message. make

                for item in items: #going product by product
                    name_tag = item.find("p", class_=lambda x: x and "productDescription" in x) #find the <p> tag(paragraph tag) with the product name
                    name = name_tag.text.strip() if name_tag else None #text strip removes extra spaces from titles, name tag helps avoid crashing if ta
                    price_tag = item.find("p", attrs={"aria-label": True}) #finds the <p> tag with the price
                    if price_tag: #did BeautifulSoup find the right element in the html?
                        raw_price = price_tag.get("aria-label") #telling it to go inside the html price_tag and get the attributes(aria label) value, wh
                        match = re.search(r"\d+\.\d*", raw_price) #\d+ is for one or more digits, \. for if there's a decimal point, \d* for if there's
                        price = float(match.group()) if match else None #the plain price without any additional signs should be gathered
                    else: #if a number wasn't found and couldn't turn into a float, say None
                        price = None

                    link_tag = item.find("a", href=True) #"a" is an anchor tag, it creates links. Only finding "a" tags that have href in it
                    link = link_tag["href"] if link_tag else None #link_tag has the whole html tag in it, asks to give the value inside the href attribu
                    products.append({"name": name, "price": price, "link": link, "sustainable": sustainable_flag}) #making each product become a row, a

                print(f"Finished page {page}") #lets me know the loop happened successfully
                success = True
                break #stops loop if it works

            #this section tells the computer to try a few times and not crash right away
            except Exception as e: #catch any error that happens. "e" holds the error message
                print(f"Retry {attempt+1} failed (page {page}): {e}") #prints a "f"ormatted string of the attempt count going up by 1, of the page and t
                time.sleep(random.uniform(5, 10)) #waits randomly between 5 and 10 seconds(random.uniform creates a random decimal number), this helps
            if not success: #if the success is still false then,
                print(f"Skipping page {page}") #print the page is being skipped, its good to skip so you can still get some data instead of stopping/crashi

            time.sleep(random.uniform(5, 8)) #waits between 5 and 8 seconds for each page switch, again mimics human behavior

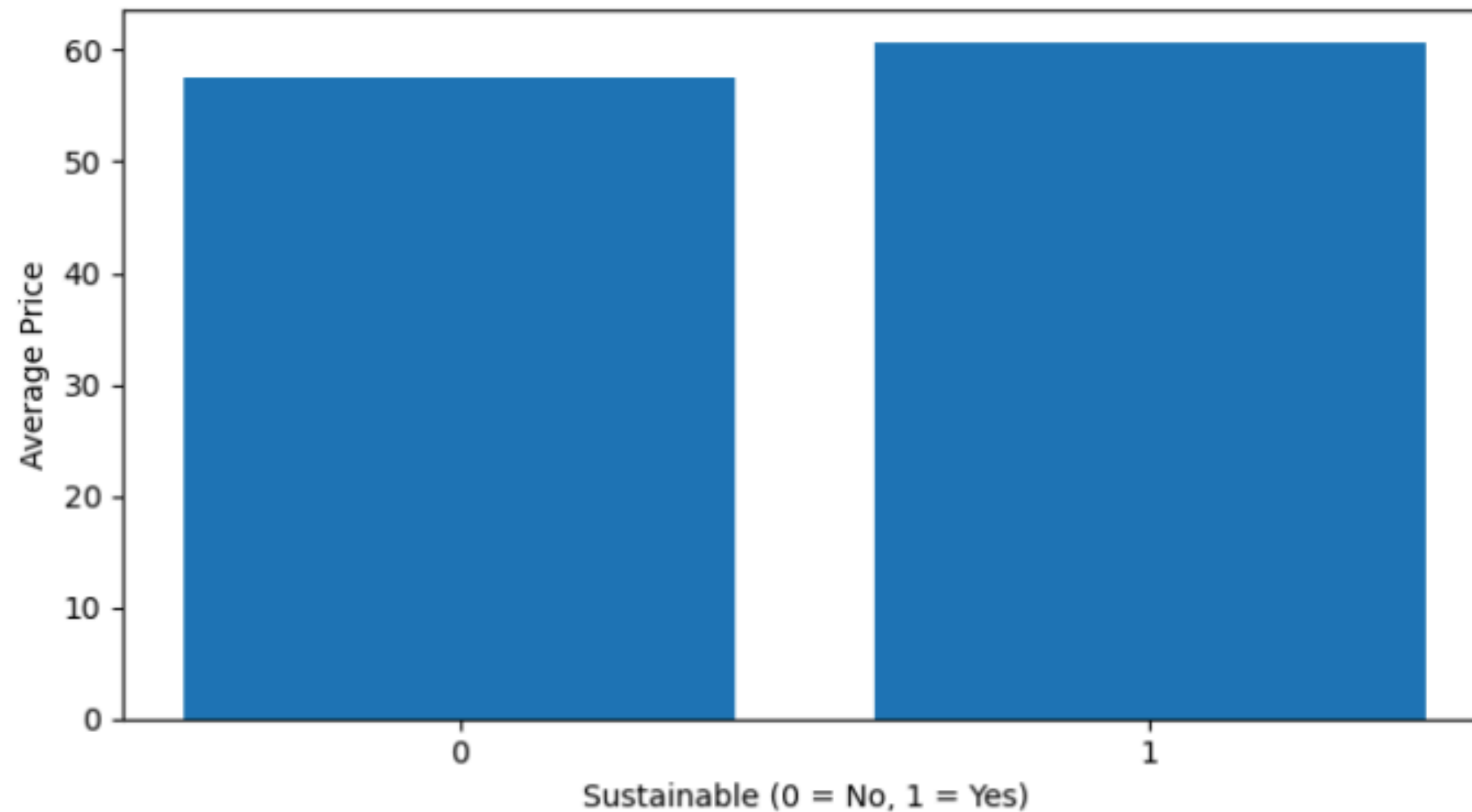
    return pd.DataFrame(products) #turns the list into a table

```


GRAPH #1

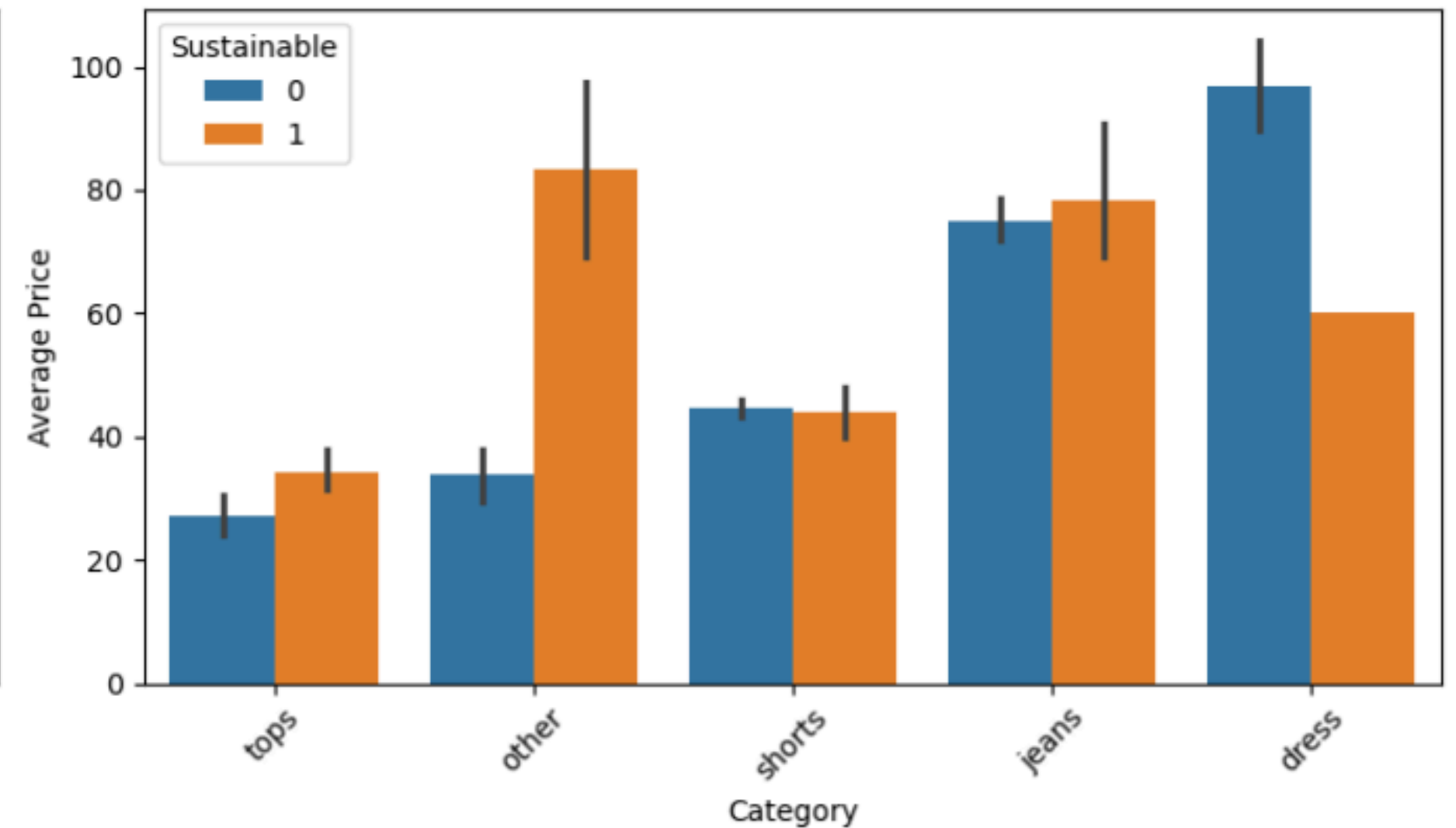
ASOS Sustainability and Pricing

Average Price:
Sustainable vs Non-Sustainable



**SUSTAINABLE PRODUCTS HAD HIGHER
AVERAGE PRICES OVERALL**

Average Price by Category and Sustainability

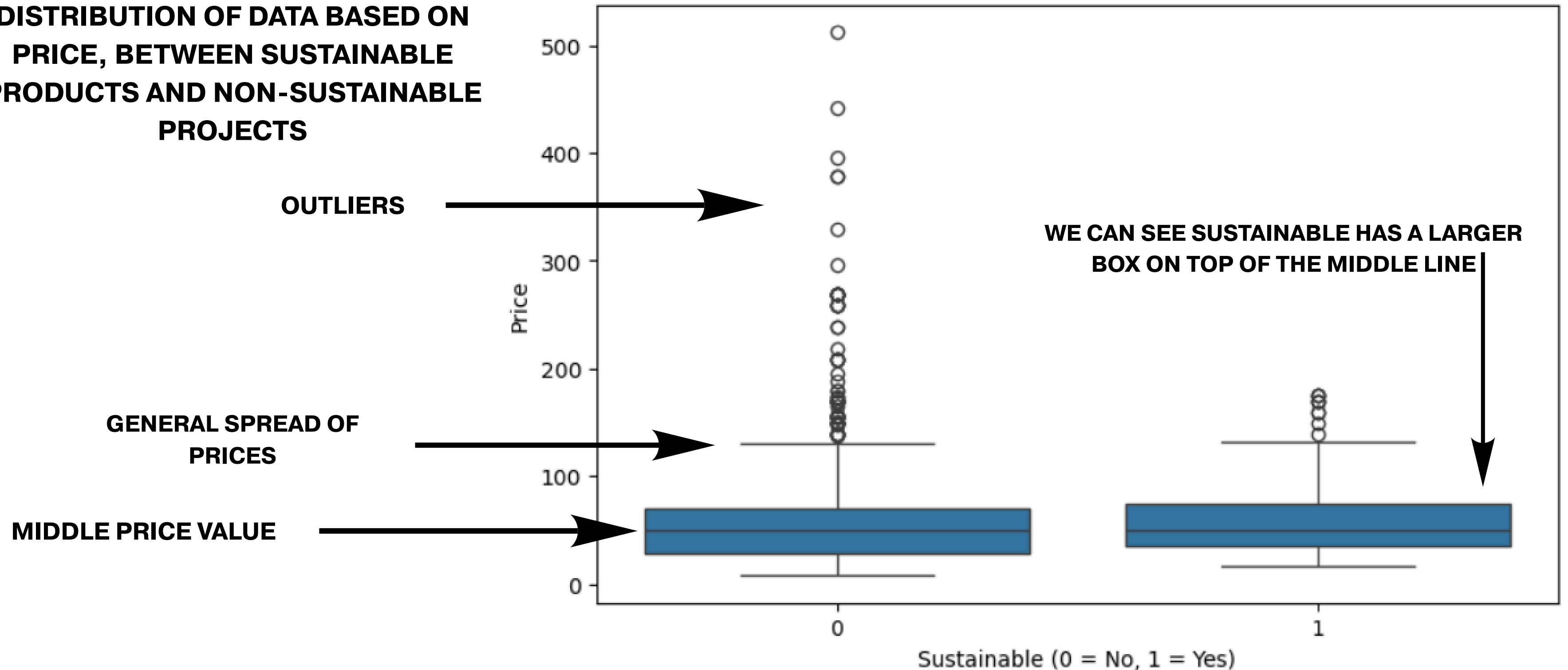


**PRICE DIFFERENCES VARIED ACROSS CATEGORIES
SOME CATEGORIES SHOWED LARGER SUSTAINABILITY
PRICES**

GRAPH #2

DISTRIBUTION OF DATA BASED ON PRICE, BETWEEN SUSTAINABLE PRODUCTS AND NON-SUSTAINABLE PROJECTS

Price Distribution by Sustainability Status



REGRESSION RESULTS

1 - SUSTAINABLE PRICES ARE ABOUT 48% HIGHER THAN NON-SUSTAINABLE PRICES ON AVERAGE, CONTROLLING FOR CATEGORY

2 - WITHIN DRESSES ONLY, SUSTAINABLE PRICES WERE ABOUT 34% LOWER ON AVERAGE THAN NON-SUSTAINABLE PRICES, BUT THE RELATIONSHIP WAS NOT STATISTICALLY SIGNIFICANT.

3 - FOR DRESSES (THE COMPARISON CATEGORY), SUSTAINABLE PRICES WERE ABOUT 34% LOWER ON AVERAGE, ALTHOUGH THE RELATIONSHIP WAS NOT STATISTICALLY SIGNIFICANT.

- SUSTAINABLE JEANS PRICES WERE ASSOCIATED WITH AN ADDITIONAL 41% INCREASE RELATIVE TO SUSTAINABLE DRESSES, BUT THE INTERACTION WAS NOT STATISTICALLY SIGNIFICANT.

ASOS Pricing Regressions			
	Reg 1 (1)	Reg 2 (2)	Reg 3 (3)
Sustainable	0.482 ^{***} (0.050)	-0.339 (0.244)	-0.339 (0.250)
Jeans	-0.205 ^{***} (0.039)		-0.187 ^{***} (0.039)
Other	-1.012 ^{***} (0.054)		-1.169 ^{***} (0.058)
Shorts	-0.669 ^{***} (0.040)		-0.678 ^{***} (0.040)
Tops	-1.411 ^{***} (0.039)		-1.423 ^{***} (0.039)
Sustainable × Jeans			0.405 (0.268)
Sustainable × Other			1.332 ^{***} (0.267)
Sustainable × Shorts			0.363 (0.431)
Sustainable × Tops			0.828 ^{***} (0.261)
Constant	4.422 ^{***} (0.030)	4.433 ^{***} (0.029)	4.433 ^{***} (0.029)
Observations	1,552	288	1,552
R ²	0.528	0.007	0.546
Adjusted R ²	0.526	0.003	0.543
Residual Std. Error	0.504 (df = 1546)	0.485 (df = 286)	0.496 (df = 1542)
F Statistic	345.836 ^{***} (df = 5; 1546)	1.930 (df = 1; 286)	205.716 ^{***} (df = 9; 1542)
Note:		p<0.1; p<0.05 ; p<0.01	

TRU ANNAFI

**THANK
YOU**

Eco590